Solving Continuous Control with Proximal Policy Optimization (PPO)

Aris Markogiannakis

March 29, 2026

Abstract

This report details the methodology and implementation of a Proximal Policy Optimization (PPO) agent designed to solve the multi-agent Unity Reacher environment. The task involves controlling 20 independent double-jointed robotic arms in a continuous action space to track moving target spheres. By utilizing a synchronous Actor-Critic architecture combined with Generalized Advantage Estimation (GAE), the agent successfully achieves the target average score of +30.0 over 100 consecutive episodes.

1 Introduction

The Reacher environment presents a continuous control problem where the state space consists of 33 dimensions (position, rotation, velocity, and angular velocities) and the action space consists of 4 continuous dimensions representing joint torques $\in [-1, 1]^4$.

While Deep Deterministic Policy Gradient (DDPG) is a common choice for continuous control, it suffers from high hyperparameter sensitivity and training instability. To ensure monotonic improvement and sample efficiency, this project implements Proximal Policy Optimization (PPO), an on-policy algorithm that constrains policy updates using a clipped surrogate objective function.

2 Methodology

2.1 Actor-Critic Architecture

The agent utilizes a shared-experience Actor-Critic architecture. Because the action space is continuous, the Actor network maps states to a probability distribution rather than discrete actions.

The Actor network computes the mean $\mu_\theta(s)$ of the actions using a fully connected network with a `tanh` activation function to bound the outputs between $[-1, 1]$. The standard deviation σ is defined as a separate, learnable state-independent parameter. The policy is thus represented as a multivariate Normal distribution:

$$\pi_\theta(a|s) \sim \mathcal{N}(\mu_\theta(s), \sigma) \tag{1}$$

The Critic network, parameterized by ϕ, maps the current state to a scalar value estimate $V_\phi(s)$ representing the expected discounted return. Both networks utilize Orthogonal Initialization to prevent vanishing/exploding gradients.

2.2 Generalized Advantage Estimation (GAE)

To balance the bias-variance tradeoff in our value estimations, we compute the Advantage function using Generalized Advantage Estimation (GAE).

First, we define the 1-step Temporal Difference (TD) error, δ_t:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \tag{2}$$

The GAE, $\hat{A}_t$, is then calculated as the exponentially weighted average of all N-step advantages, controlled by the smoothing parameter λ:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \tag{3}$$

By computing this recursively backwards through the sampled trajectory, we efficiently obtain high-quality advantage targets for the Actor network.

2.3 The PPO Objective

PPO prevents destructively large policy updates by clipping the probability ratio of the new policy against the old policy. The probability ratio is defined as:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \tag{4}$$

The clipped surrogate objective for the Actor is:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{5}$$

where ϵ is the clipping hyperparameter (e.g., 0.2).

The Critic network is updated by minimizing the Mean Squared Error (MSE) between its predictions and the empirical returns ($V_t^{target} = \hat{A}_t + V_\phi(s_t)$):

$$L^{VF}(\phi) = (V_\phi(s_t) - V_t^{target})^2 \tag{6}$$

To encourage exploration, an entropy bonus $S[\pi_\theta]$ is added. The final master loss function maximized at each update step is:

$$L_{PPO}^{TOTAL} = \hat{\mathbb{E}}_t \left[L^{CLIP}(\theta) - c_1 L^{VF}(\phi) + c_2 S[\pi_\theta](s_t) \right] \tag{7}$$

3 Hyperparameters

The following hyperparameters were used to solve the environment:

- **Discount Factor (γ)**: 0.99
- **GAE Parameter (λ)**: 0.95
- **Clipping Parameter (ϵ)**: 0.2
- **PPO Epochs**: 10
- **Mini-batch Size**: 256
- **Learning Rate (Actor & Critic)**: 3×10^{-4}
- **Value Loss Coefficient (c_1)**: 0.5
- **Entropy Coefficient (c_2)**: 0.01

4 Results

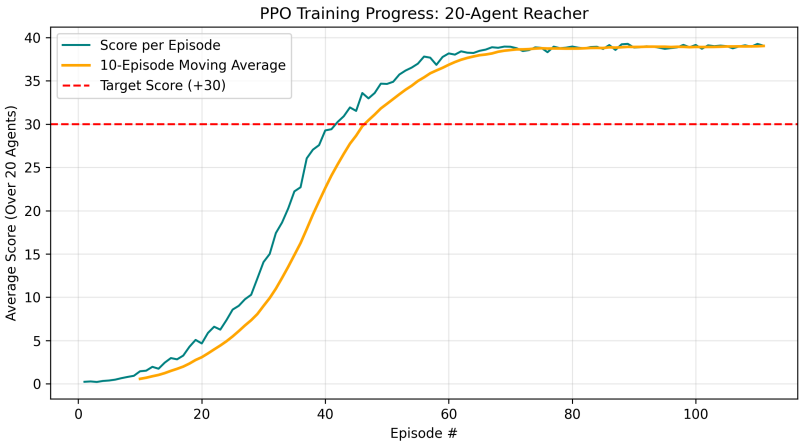

Figure 1: Training progress showing the moving average score crossing the +30 threshold.

The 20-agent PPO implementation demonstrated highly stable learning. The agent successfully crossed the target threshold of an average score of +30.0 over 100 consecutive episodes.

5 Conclusion and Future Work

PPO proved to be a robust and sample-efficient algorithm for multi-agent continuous control. Future work could explore integrating continuous action spaces with Soft Actor-Critic (SAC) to observe differences in sample efficiency, or adding Recurrent Neural Networks (LSTMs) to the Actor-Critic heads to handle environments with partial observability.